SDS - REQ-NF-004 시각 주입으로 시간 고정 테스트 지원

작성일 2026-10-17 · 근거: 요구사항정의서_엑셀양식_v3_1_3.xlsx (F열) · SRS 없음 · 대상: D:₩legacy-portal

1. 개요

요구사항 ID	REQ-NF-004
요구사항명	시각 주입으로 시간 고정 테스트 지원
대 / 중 / 소분류	PORTAL / Quality / Clock
우선순위	Low
진행현황	개발중 (비고: 리팩토링 백로그 R13)
요약	업무 코드의 `LocalDateTime.now()` 직접 호출을 주입된 `Clock` 으로 바꿔 테스트에서 시각을 고정한다. 결재 · 공지 서비스와 감사로그 기록기는 이미 Clock 을 주입받으며(작업 트리에 반영, 아직 미커밋), 운영 소스에 인자 없는 `now()` 호출이 남아 있지 않다. 일정 서비스만 Clock 을 받지 않는다(현재 시각을 쓰는 곳이 없음). 이 변경은 동작을 바꾸지 않는 **리팩토링**(숨은 의존성 주입)이다.

2. 근거

SRS 없음 (엑셀 F열 · 소스만으로 분석). 아래는 요구사항정의서 F열(상세내용) 원문이다.

```
[개요]
업무 코드가 LocalDateTime.now() 를 직접 부르지 않고 주입된 Clock 을 써서, 테스트에서 시각을 고정할 수 있게 한다.

[상세 기능 요구사항]
1) 결재 · 공지 · 일정 서비스와 감사로그 기록기가 Clock 빈을 주입받는다.
2) 운영에서는 시스템 기본 시간대 Clock 을 쓴다.
3) 테스트에서는 고정 Clock 으로 createdAt · updatedAt · 감사로그 시각을 정확히 비교한다.

[입력 / 출력]
- 입력: Clock 빈
- 출력: 고정 가능한 현재 시각

[제약 조건 / 비기능 요구사항]
- 운영 동작(기록되는 시각 의미 · 형식)을 바꾸지 않는다.
- public 시그니처를 바꾸지 않는다.

[검증 기준]
- 고정 Clock 테스트에서 기안 응답의 createdAt 이 고정 시각과 같다.
- 소스에 LocalDateTime.now() 직접 호출이 남지 않는다.

[작업 대상]
legacy-portal (D:₩legacy-portal) Spring Boot 3.2 / Java 21
src₩main₩java₩kr₩hull₩portal₩approval₩ApprovalService.java
src₩main₩java₩kr₩hull₩portal₩notice₩NoticeService.java
src₩main₩java₩kr₩hull₩portal₩common₩ConsoleAuditLogger.java

[Worker Settings]
- MainWorker          : claude-main
```

- SubWorker : codex-critic
- MainWorker-SubAgent : backend-expert

3. 분석 결과

3.1 추적표

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
상세 1	결재 · 공지 · 일정 서비스와 감사로그 기록기가 Clock 빈을 주입받는다.	SRS 없음	approval/ApprovalService.java:56, 66-74, notice/NoticeService.java:46, 55-63, common/ConsoleAuditLogger.java:32, 39-42, schedule/ScheduleService.java:34-38	부분 충족	결재 · 공지 · 감사로그 기록기는 충족. **ScheduleService 는 Clock 을 받지 않는다** - 일정에는 생성 · 수정 시각 필드가 없고 now() 호출도 없어(Schedule.java:31-36) 실질 영향은 없다. 일정의 감사로그 시각은 주입된 ConsoleAuditLogger 의 Clock 을 따른다.
상세 2	운영에서는 시스템 기본 시간대 Clock 을 쓴다.	SRS 없음	common/ClockConfig.java:18-24	충족	@Bean Clock clock() = Clock.systemDefaultZone().
상세 3	테스트에서는 고정 Clock 으로 createdAt · updatedAt · 감사로그 시각을 정확히 비교한다.	SRS 없음	src/test/.../approval/ClockInjectionTest.java:56-104	충족	기안 생성 · 상신 수정시각 · 감사로그 문자열 · 공지 작성시각을 고정 시각과 정확히 비교.
입력	Clock 빈	SRS 없음	ClockConfig.java:21-24	충족	
출력	고정 가능한 현재 시각	SRS 없음	ApprovalService.java:81, 140, 149, 158, 168, NoticeService.java:67, ConsoleAuditLogger.java:55	충족	모든 현재 시각 사용처가 LocalDateTime.now(clock).
제약 1	운영 동작(기록되는 시각 의미 · 형식)을 바꾸지 않는다.	SRS 없음	ClockConfig.java:23, ConsoleAuditLogger.java:30, 54-57	충족	시스템 기본 시간대(레거시 now() 와 같은 기준), 포맷 yyyy-MM-dd HH:mm:ss 그대로.
제약 2	public 시그니처를 바꾸지 않는다.	SRS 없음	ApprovalService.java:59-63, NoticeService.java:49-53, ConsoleAuditLogger.java:34-36	충족	기존 생성자 · public 메서드는 그대로 두고 Clock 을 받는 생성자를 **추가**만 했다(두 생성자 모두 @Autowired(required = false)).

3.2 판정 요약

충족 6 / 부분 충족 1 / 미충족 0 / 불일치 0 / 확인 불가 0 (총 7항목)

3.3 검증 기준 ↔ 테스트

검증 기준(F열)	테스트	비고
고정 Clock 테스트에서 기안 응답의 createdAt 이 고정 시각과 같다.	`ClockInjectionTest#고정_Clock이면_기안의_생성시각과_수정시각이_고정_시각이다`	**서비스가 돌려준 엔티티로 비교한다. HTTP 응답 JSON(`ApprovalResponse.createdAt`) 단계의 고정 시각 비교 테스트는 없다(9장 제안).**
소스에 LocalDateTime.now() 직접 호출이 남지 않는다.	`RequirementUpdateAcceptanceTest#운영_소스에_LocalDateTime_now_직접_호출이_남아_있지_않다`	**LocalDateTime · LocalDate · LocalTime · Instant 의 인자 없는 now() 를 주석 제외 후 검사. Grep 결과도 `now(clock)` 뿐.**
(상세 2) 운영 Clock = 시스템 기본 시간대	`RequirementAcceptanceTest#앱의_Clock_빈은_시스템_기본_시간대다`	
(상세 3) 감사로그 · 공지 시각	`ClockInjectionTest#고정_Clock이면_감사로그_시각이_고정_시각이다`, `#고정_Clock이면_공지_작성시각이_고정_시각이다`, `#고정_Clock이면_상신_후_수정시각도_고정_시각이다`	

4. 구성 요소 설계

클래스	책임	주요 시그니처 (현행 유지)	상태
`common.ClockConfig`	현재 시각 출처 빈	`@Bean public Clock clock()` → `Clock.systemDefaultZone()`	구현됨(미커밋)
`approval.ApprovalService`	기안 · 처리 시각	`ApprovalService(ApprovalRepository, UserRepository, ApprovalNotifier, ApprovalAuditRecorder)` [기존], `ApprovalService(..., Clock clock)` [추가]	구현됨
`notice.NoticeService`	공지 작성 시각	`NoticeService(NoticeRepository, UserRepository, MailSender, AuditLogger)` [기존], `NoticeService(..., Clock clock)` [추가]	구현됨
`common.ConsoleAuditLogger`	감사로그 시각	`ConsoleAuditLogger()` [기존], `ConsoleAuditLogger(Clock clock)` [추가]	구현됨
`schedule.ScheduleService`	일정 흐름	`ScheduleService(ScheduleRepository, UserRepository, AuditLogger)`	Clock 미주입 - **설계안**: 현재 시각 사용처가 생길 때(예: 생성 시각 컬럼 추가) 같은 방식(기존 생성자 유지 + Clock 생성자 추가)으로 주입. 지금 넣으면 쓰지 않는 필드(Sonar 경고)가 되므로 F열 문구와 실익 중 선택 필요 → 10장.

생성자 선택 규칙: Clock 빈이 있는 컨텍스트(앱 · `@SpringBootTest`)는 Clock 생성자, 없는 슬라이스(`@DataJpaTest`)는 기존 생성자를 스프링이 고른다(`ApprovalService.java:43-44` 주석).

5. 인터페이스 설계

REST API · 요청/응답 JSON 변경 없음. `createdAt` · `updatedAt` 은 기존과 같이 `LocalDateTime`(ISO-8601 문자열)으로 직렬화되며 값의 의미(시스템 기본 시간대의 현재)도 같다. 감사로그 출력 형식 `[AUDIT] [yyyy-MM-dd HH:mm:ss] {동작} id={id} by={userId}` 그대로.

6. 데이터 설계

스키마 · 저장값 변경 없음. 시각 컬럼: `Approval.createdAt` · `updatedAt`(`Approval.java:59-60`), `Notice.createdAt`(`Notice.java:38`). 결재 생성 시 두 값은 같은 순간(`Approval.java:111-112`). 상태 code(0/1/2/3/9)와 무관.

7. 처리 흐름

단계	운영	테스트
1 빈 생성	`ClockConfig.clock()` = 시스템 기본 시간대	`Clock.fixed(2026-10-17T09:30, systemDefault)` 를 직접 new 로 주입
2 시각 획득	`LocalDateTime.now(clock)` (서비스 → 엔티티 팩토리/행위에 인자로 전달)	
3 저장	createdAt · updatedAt 저장	
4 감사로그	`ConsoleAuditLogger.format` 이 같은 방식으로 시각 획득 · 포맷	

8. 예외 · 경계 처리

경우	동작	레거시 보존 여부
Clock 빈 없는 슬라이스 테스트	기존 생성자 → `Clock.systemDefaultZone()`	보존
무시된 처리	엔티티가 false 를 돌려주면 updatedAt 불변 (`ApprovalProcessCharacterizationTest#무시된_처리는_수정시각을_바꾸지_않는다`)	보존
시간대	운영 = 시스템 기본. 서버 시간대가 바뀌면 기록 시각도 바뀜(레거시와 동일)	보존

9. 검증 설계

현재 검증 기준 2개 모두 대응 테스트가 있다. 기준 1을 문구 그대로("기안 **응답**") 확인하는 테스트를 추가 제안한다.

```
@Test void 고정_Clock이면_기안_API_응답의_createdAt이_고정_시각이다()
@Test void 고정_Clock이면_공지_게시_감사로그_시각이_고정_시각이다()
@Test void 고정_Clock이면_일정_등록_감사로그_시각이_고정_시각이다()

// 예: @SpringBootTest + @Import(FixedClockTestConfig.class) 에서 Clock 빈을 고정값으로 교체
private static final LocalDateTime FIXED_TIME = LocalDateTime.of(2026, 10, 17, 9, 30, 0);
mockMvc.perform(post("/api/approvals").contentType(MediaType.APPLICATION_JSON).content(createRequestJson)
)
    .andExpect(jsonPath("$.createdAt").value("2026-10-17T09:30:00"));
```

10. 검토 의견 · 후속 과제

구분	내용
보존 대상	기록 시각의 의미(시스템 기본 시간대의 지금), 감사로그 형식, public 메서드 시그니처, 기존 생성자, 응답 JSON 필드, DB 저장값. 특성화 테스트는 수정 없이 green 이어야 한다.
개선 대상	ScheduleService 의 Clock 주입 여부를 결정해야 한다(F열 문구는 "일정 서비스"를 포함하지만 현재 시각 사용처가 없음). 권장: 요구사항 문구를 "현재 시각을 쓰는 서비스"로 고치거나, 일정 생성 시각 컬럼 추가 요구가 생길 때 함께 주입.
개선 대상	변경분(`ClockConfig.java` 신규, `ApprovalService` · `NoticeService` · `ConsoleAuditLogger` 수정, `ClockInjectionTest` 신규)이 아직 커밋되지 않았다(git status). 전체 테스트 green 확인 후 "한 기법 한 커밋" 원칙으로 커밋 필요.
개선 대상	생성자 2개 + `@Autowired(required = false)` 조합은 스프링의 "가장 많이 충족되는 생성자" 선택 규칙에 기대고 있다. 장기적으로는 Clock 빈을 테스트 슬라이스에도 제공(`@Import(ClockConfig.class)`)하고 생성자를 하나로 줄이는 것을 검토.
연계 요구사항	없음(독립). 감사로그 형식은 결재 · 공지 · 일정 공통.